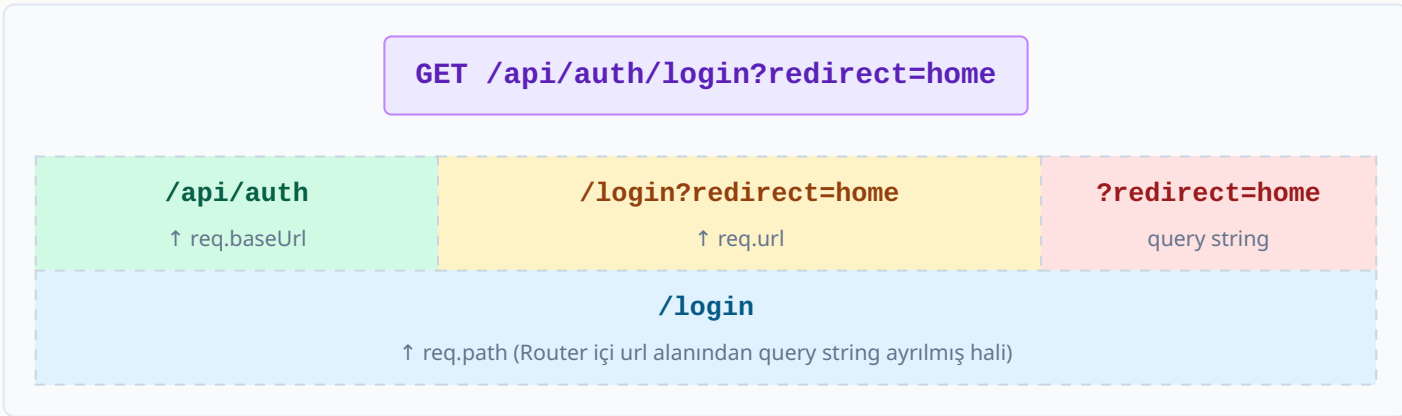


Express.js Request URL Alanları ve Router Mimarisi

Gelişmiş Backend Geliştirme İçin İstek Parametreleri ve URL Ayırıştırma Kılavuzu

Express.js mimarisinde, istemciden gelen bir HTTP isteğinin (req) hangi alt rotaya (Router) uğradığı ve URL yapısının nasıl parçalandığı, özellikle büyük ölçekli ve çok katmanlı kurumsal uygulamalarda kritik önem taşır. Aşağıda, görseldeki yapının detaylandırılmış analizi ve gelişmiş kullanım senaryoları yer almaktadır.

1. Yapısal URL Ayırıştırma Şeması



2. Alan Tanımları ve Teknik Karşılıkları

Alan (Property)	İçerik ve Davranış Açıklaması	Örnek Değer (Yukarıdaki Senaryo)
req.originalUrl	İstemcinin uygulamaya gönderdiği ham ve tam URL adresidir. Alt router yapılarına girildiğinde asla değişmez . Uygulama genelinde loglama ve audit işlemleri için en güvenli alandır.	/api/auth/login?redirect=home
req.baseUrl	İsteğin ana uygulamadan ilgili alt Router (mini-app) yapısına yönlendirilirken eşleştiği kök (mount) yolunu (prefix) belirtir. Ana app seviyesinde boş bir string ("") döner.	/api/auth
req.url	İsteğin, yönlendirildiği aktif Router içindeki göreceli (relative) yoludur. Query string parametrelerini de içerir . Middleware katmanlarında dinamik rewrite işlemlerine izin verir.	/login?redirect=home
req.path	İsteğin yönlendirildiği aktif Router içindeki temiz, parametresiz (query string hariç) halidir. Sadece rotalama ve endpoint eşleştirme süreçlerinde baz alınır.	/login

💡 Kritik Bilgi: **req.url** Değişebilirliği

Express.js mimarisinde **req.url** alanı yazılabilir (writable) bir alandır. Bir middleware içerisinde bu alanı değiştirirseniz, Express sonraki routing adımlarını yeni verdiğiniz değere göre yürütür. Ancak **req.originalUrl** değişmez, ilk halini korur.

3. Uygulamalı Kod Örnekleri ve Router Kurgusu

Aşağıdaki kod blokları, yukarıda şematize edilen yapının Express.js projesinde nasıl ayağa kaldırılacağını ve alt yönlendiricilerin (sub-routers) bu verileri nasıl manipüle ettiğini göstermektedir.

Ana Uygulama Dosyası (app.js / server.js)

```
const express = require('express');
const app = express();
const authRouter = require('./routes/authRouter');

// Ara katman (Middleware): Ana uygulama seviyesindeki değerler
app.use((req, res, next) => {
  console.log('--- Ana App Seviyesi ---');
  console.log('originalUrl:', req.originalUrl); // "/api/auth/login?redirect=home"
  console.log('baseUrl:', req.baseUrl);        // "" (Çünkü henüz bir router'a girmedi)
  console.log('url:', req.url);                 // "/api/auth/login?redirect=home"
  next();
});

// Alt Router'ın "/api/auth" prefix'i ile bağlanması (Mounting)
app.use('/api/auth', authRouter);

app.listen(3000);
```

Alt Yönlendirici Dosyası (routes/authRouter.js)

```
const express = require('express');
const router = express.Router();

// Hedef Endpoint: GET /api/auth/login
router.get('/login', (req, res) => {
  console.log('--- Alt Router (Auth) Seviyesi ---');

  const urlMetrics = {
    originalUrl: req.originalUrl, // /api/auth/login?redirect=home
    baseUrl: req.baseUrl,        // /api/auth
    url: req.url,                // /login?redirect=home
    path: req.path,              // /login
    query: req.query              // { redirect: 'home' }
  };

  res.json({
    success: true,
    metrics: urlMetrics
  });
});

module.exports = router;
```

4. İleri Seviye Senaryolar ve Fark Yaratan Geliştirici Bilgileri

Senaryo A: Çok Katmanlı İççe Router Kullanımı (Nested Routers)

Eğer `app.use('/api', apiRouter)` ve ardından `apiRouter.use('/auth', authRouter)` şeklinde iççe bir kurgu varsa; `authRouter` içerisine ulaşıldığında `req.baseUrl` değeri **/api/auth** olacaktır. Yani kat edilen tüm mount yolları birleşir.

Senaryo B: Güvenli Audit Loglama (Sistem Günlüğü) Middleware Yazımı

Sisteminizde gelen tüm isteklerin performansını ve rotasını loglayan bir global middleware yazarken kesinlikle `req.url` kullanmamalısınız. Çünkü dinamik alt rotalar url alanını değiştirebilir veya temizleyebilir. Analiz araçları için değişmez tek kaynak **`req.originalUrl`**'dir.